CBSE CLASS X • INFORMATION TECHNOLOGY (SUBJECT CODE 402)

UNIT 2: ELECTRONIC SPREADSHEET (ADVANCED) — CHAPTER 5

USING MACROS IN SPREADSHEET — STRUCTURED NOTEBOOK & REVISION NOTES

Syllabus Sub-Unit 2 Complete Theory • Recording & Running • Basic IDE & API • Macro Functions • Zero Over-Shortening

---

1. Concept, Definition and Core Advantages of Macros

Definition: In LibreOffice Calc, a **Macro** is a saved, named sequence of commands, keystrokes, and operations that are stored for later reuse. When executed, a macro automatically replays the exact recorded sequence of actions to perform a task.

- **Task Automation:** Automates repetitive keystrokes, cell formatting, data entry, and multi-step calculations across multiple worksheets without manual re-entry.
- **Error Reduction:** Substantially minimizes the risk of human errors and typographical mistakes that naturally increase during monotonous repetitive tasks.
- **Time Efficiency:** Dramatically speeds up recurring computing tasks (e.g., applying identical layout styling to 48 section result sheets in a single click).
- **Simplifying Complex Computations:** Consolidates intricate mathematical or structural operations under a single callable name or customized user function.
- **Crucial Recording Limitation:** The Macro Recorder logs keyboard typing, menu commands, and dialog box entries, but **does NOT record mouse movements** across the window.
- **Disabled by Default:** For security reasons (to prevent malicious macro scripts from executing automatically), macro recording is disabled by default in LibreOffice Calc and must be turned on manually.
- **Macro Security Caution:** Macros from untrusted or unknown external sources should never be enabled, as they can inject malware or corrupt spreadsheet data.

2. Enabling Macro Recording in LibreOffice Calc

Before recording any macro, the feature must be enabled through the application options dialog:

Step 1: Choose **Tools** → **Options...** from the Menu bar (or press the keyboard shortcut **Alt + F12**).

Step 2: In the Options dialog box, click the expand arrow next to **LibreOffice** in the left pane and select the **Advanced** sub-entry.

Step 3: On the right side under the **Optional Features** section, check the checkbox labelled **'Enable macro recording (may be limited)'**.

Step 4: Click **Apply**, and then click **OK**. The macro recording feature is now fully activated.

3. Recording and Running Macros (Complete Practical Procedures)

A. Step-by-Step Procedure to Record a Macro:

Step 1: Open the spreadsheet containing the data where the repetitive task is required.

Step 2: Select the cell range to format or manipulate (e.g., header range *B1:G5*).

Step 3: Choose **Tools** → **Macros** → **Record Macro** from the Menu bar.

Step 4: Calc begins recording actions and displays a small floating **Record Macro** toolbar with a single button: **Stop Recording**.

Step 5: Perform your intended actions (e.g., change font size to 12, make text bold, set cell background color, or enter standard formulas). All valid actions are recorded.

Step 6: When finished, click the **Stop Recording** button on the floating toolbar.

Step 7: The **LibreOffice Basic Macros** dialog box appears on the screen.

Step 8: In the *Save Macro In* left tree, click the **+** icon next to **My Macros**, expand the **Standard** library, and select **Module1** (the default container for user-created macros).

Step 9: In the **Macro Name** box at the top, replace the default name (*Main*) by typing a descriptive, legal name (e.g., *FormatHeaders*).

Step 10: Click **Save**. The macro is stored in Module1 and ready for reuse.

B. Rules for Naming a Macro (CBSE Legal vs. Illegal Identifiers):

- **1. Must Begin with an Alphabet:** The first character must always be a letter (e.g., A-Z or a-z). It cannot begin with a number or symbol.
- **2. No Blank Spaces Allowed:** Macro names cannot contain spaces (e.g., Format Table is invalid; use FormatTable or Format_Table).
- **3. No Special Characters Except Underscore:** Punctuation marks, arithmetic symbols, and special characters (*, @, #, -, $, %, ?) are strictly prohibited. The **underscore (_)** is the only allowed symbol.
- **4. Legal Examples:** myMacro, Sort_Marks, CalculateTotal1, FormatHeading.
- **5. Illegal Examples:** solution*word (contains asterisk), my macro (contains space), 2ndMacro (starts with digit), test@calc (contains special character).

C. Step-by-Step Procedure to Run an Existing Macro:

Step 1: Open the worksheet and select the target cell range where you want to execute the macro.

Step 2: Choose **Tools** → **Macros** → **Run Macro...** from the Menu bar.

Step 3: In the **Macro Selector** dialog box, expand **My Macros** → **Standard** → **Module1** in the *Library* list on the left.

Step 4: Select your macro name from the **Macro Name** listbox on the right (e.g., *FormatHeaders*).

Step 5: Click the **Run** button. Calc instantly applies all saved actions to the selected range.

AS. NOTES IT-402

4. Internal Storage, Code Architecture & the LibreOffice Basic IDE

Calc translates every macro into **LibreOffice Basic** code, managed inside the built-in **Integrated Development Environment (IDE)**.

- **Accessing Basic IDE:** Choose **Tools** → **Macros** → **Organize Macros** → **LibreOffice Basic...** (or click *Edit Macros*). Select a module and click **Edit** (or press **F5** to run code inside the IDE).
- **Language Characteristics:** LibreOffice Basic is **case-insensitive** (keywords and variable names can be typed in uppercase, lowercase, or mixed case).
- **Comments Syntax:** Non-executable explanatory comments in code begin with the keyword Rem (Remark) or a single quotation mark (').

Subroutine Macro vs. Function Macro (Structural Comparison):

Comparison Parameter	Subroutine Macro (Sub ... End Sub)	Function Macro (Function ... End Function)
Enclosing Statements	Begins with Sub <name> and ends with End Sub.	Begins with Function <name> and ends with End Function.
Return Value	**Cannot return a value** to a calling formula cell.	**Returns a computed value** directly to the calling cell formula.
Invocation Method	Run via menu (Tools → Macros → Run Macro...), toolbar buttons, or hotkeys.	Called directly inside a cell formula like built-in functions (e.g., =Addres()).
Result Assignment	Executes imperative commands directly on document objects.	The final computed result **must be assigned to the function name**.

5. Creating Macros as Functions & Passing Arguments

A macro function acts as a custom user-defined formula for repetitive calculations where no built-in Calc function exists. It is declared between Function and End Function statements, and the return value is assigned to the function name.

- **Example 1: Parameterless Macro Function (Standard Text Insertion):**
  ```
  Function Addres()
    Addres = "123, ABC Road, T Block, X Nagar, Delhi, India"
  End Function
  ```
  Usage in Spreadsheet: Enter =Addres() into any blank cell; Calc automatically fills the address text.
- **Example 2: Macro Function Accepting Arguments:**
  Values passed inside parentheses when calling a function are called its **Arguments**:
  ```
  Function ValueDoubled(Optional val)
    ValueDoubled = val * 2
  End Function
  ```
  Usage in Spreadsheet: =ValueDoubled(25) returns 50. =ValueDoubled(C16) reads the value of cell C16 (say, 17) and returns 34.
- **CRITICAL RULE — Arguments Passed as Values:**
  In LibreOffice Calc, **arguments passed to a macro from a cell formula are always passed as VALUES**, not as cell references. The macro receives only the raw evaluated number/text and has no internal way of knowing which physical cell coordinates provided that data.

6. Accessing & Manipulating Cell Values Directly via Basic API

To manipulate cells programmatically using code rather than toolbar recording, developers use the Universal Network Objects (UNO) object model:

- **1. Declare Object Variables:** Use the Dim statement to declare variables:
  `Dim myDoc As Object, mySheet As Object, myCell As Object`
- **2. Reference Active Document:** Bind the document variable to the open workbook using ThisComponent:
  `myDoc = ThisComponent`
- **3. Reference Active Sheet:** Access the sheet via controller or zero-based index:
  `mySheet = myDoc.CurrentController.getActiveSheet()` *(or myDoc.Sheets(0))*
- **4. Reference Specific Cell:** Fetch the desired cell object by its coordinate label string:
  `myCell = mySheet.getCellRangeByName("C5")`
- **5. Read Cell Value:** Use the .getValue() method to read numbers:
  `val = myCell.getValue()`
- **6. Write Cell Value:** Assign data using .setValue(<number>) or .Value = <number>:
  `myCell.setValue(75000)`
- **Worked Example — Calculating 10% Discount from Cell B4 into Cell C4:**
  ```
  Sub CalculateDiscount()
    Dim myDoc As Object, mySheet As Object, cellAmt As Object, cellDisc As Object
    myDoc = ThisComponent
    mySheet = myDoc.CurrentController.getActiveSheet()
    cellAmt = mySheet.getCellRangeByName("B4")
    cellDisc = mySheet.getCellRangeByName("C4")
    cellDisc.Value = cellAmt.getValue() * 0.10
  End Sub
  ```

A.S. NOTES IT-402

7. Sorting Columns Using a Macro & Critical Constraints

- **Procedure:** Start recording (**Tools** → **Macros** → **Record Macro**) → Select data range (e.g., *B38:C46*) → Choose **Data** → **Sort...** → Choose Sort Key column (e.g., *Marks*) and order (Ascending/Descending) → Click **OK** → Click **Stop Recording** on floating toolbar → Save macro in *Module1* as Sort_Marks.
- **Identical Range Mandate:** A recorded sorting macro functions successfully **ONLY if the target data resides in the EXACT identical cell range** (e.g., *B38:C46*) across sheets.
- **Why It Fails on Different Ranges:** When recording a sort, Calc hardcodes the specific coordinates of the selected range into the internal Basic commands. If run on a different range (e.g., *D10:E25*), it will either sort the original hardcoded range or produce errors.

A.S. NOTES IT-402